

Architetture e Sistemi Operativi

Contents

1	Introduzione	3
2	Processore, memoria e istruzioni	4
2.1	Processore	4
2.2	Memoria	4
2.3	Tipi di istruzioni	4
3	Livelli di astrazione dei calcolatori	6
3.1	Principi importanti	6
4	Circuiti Digitali	8
4.1	Sistema binario	8
4.2	Numeri negativi	8
4.3	Numeri con la virgola (Floating Point)	9
4.4	Caratteri (ASCII)	9
4.5	Notazione per grandezza input/output	10
4.6	Tempi per calcolare risultati	10
5	Esadecimale	11
5.1	Processo di progettazione dei circuiti digitali	11
5.2	Mappe di Karnaugh	12
6	Moduli	13
6.1	Sommatore	13
6.2	Sottrattore (sommatore in complemento a 2)	13
6.3	Comparatore	13
6.4	Codificatore	13
6.5	Moduli per algebra booleana	13
6.6	Modulo di scelta	14
6.7	Full Adder	14
6.8	Multiplexer	15
6.9	Demultiplexer	15

1 Introduzione

In questo corso andremo a:

- Capire come funziona un elaboratore.
- Capire quali tipi di istruzioni è in grado di eseguire un elaboratore nativamente.
- Come lo si può dotare di un sistema operativo.

Il corso utilizza Windows e Linux come esempi di sistemi operativi.

Codice Teams: 6uk5ntg

Email Docente: marco.delutto@unipi.

2 Processore, memoria e istruzioni

2.1 Processore

Il **processore** è in grado di fare operazioni piccole.

Il processore contiene i **registri**, memorie piccolissime e velocissime utilizzate per memorizzare dati temporanei legati alle istruzioni da eseguire. Ecco alcuni registri speciali:

- **Program Counter (PC)**: un registro speciale che conserva lo stato di esecuzione memorizzando l'indirizzo della prossima istruzione da eseguire.
- **Instruction Register (IR)**: contiene l'istruzione prelevata dalla memoria che deve essere eseguita.
- **Stack Pointer (SP)**: contiene la posizione attuale dello stack.

2.2 Memoria

Il processore è unito ad una **memoria**, costituita da **celle**, è vista come un vettore:

$$M[i] = k$$

Ogni cella ha un valore e la sua capacità è 1 parola (32 o 64 bit).

Una volta che il processore viene acceso, esso esegue:

```
while(true):  
    instr = getInstructionFrom(M)  
    decoded = decode(instr)  
    execute(decoded)  
    determineResults()  
    ...
```

Il processore quindi:

- Estrae istruzioni dalla memoria
- Le decodifica
- Le esegue
- Calcola i risultati

2.3 Tipi di istruzioni

Le istruzioni possono essere:

- Operative
 - Aritmetiche
 - * ADD
 - * SUB
 - * MUL

- * DIV
 - Logiche
 - * AND
 - * OR
 - * NOT
 - Shift / rotazione
 - * SHIFT
 - * ROT
 - ...
- Di Memoria
 - LOAD
 - * Restituisce un valore da un indirizzo:

LOAD val , addr
 - STORE
 - * Salva un valore in un indirizzo:

STORE val , addr
- di Salto
 - "Saltano" pezzi di codice, volendo anche secondo istruzioni, aggiornando il PC alla corrispondente istruzione.

```
inizio :  
  \dots  
  jmp fine :  
  —skipped code—  
fine :
```

3 Livelli di astrazione dei calcolatori

Per capire come opera un computer osserviamo dei livelli di astrazione, sviluppati a "torre" nel seguente modo:

applicazioni
sistema operativo
architettura (set istruzioni)
 μ - architettura (processore)
logica (componenti)
circuiti digitali (0/1, 5V/0V)
circuiti analogici
dispositivi
fisica

Non andremo a vedere niente delle caratteristiche hardware degli ultimi 3 livelli, non ci interessa.

Per capire:

- Noi partiamo col realizzare dei circuiti digitali che servono per creare dei componenti
- Tali componenti ci servono per realizzare un processore
- Al processore dovremo dare in pasto delle istruzioni
- Le istruzioni servono al sistema operativo e alle applicazioni

Tra i livelli di "architettura" e "sistema operativo" si trova un **interfaccia**, genericamente attribuita al livello **i**. Questo significa che:

- Il livello del sistema operativo corrisponde al livello **i** (si trova al livello dell'interfaccia)
- Il livello dell'architettura corrisponde al livello **i - 1**

Al livello **i + 1**, tra sistema operativo e applicazioni, si trovano le **chiamate al sistema operativo**, chiamate **API**.

Ogni livello conosce solo il **livello inferiore** a se, questo è per mantenere i livelli superiori attraverso diversi livelli inferiori; per esempio un telefono Samsung e un Motorola sono realizzati in maniera diversa, ma entrambi utilizzano Android, questo è possibile proprio grazie al fatto che i livelli non si concentrano sui livelli soprastanti.

Un altro esempio: non si possono portare app iOS su Android.

Spesso esistono dei traduttori che permettono di convertire istruzioni (x64,x86) per permettere compatibilità con sistemi operativi e processori differenti.

3.1 Principi importanti

I componenti che svolgono le istruzioni seguono questi concetti:

- **Gerarchia**: il principio dei livelli, ogni livello utilizza i componenti dei livelli sottostanti senza occuparsi di cosa fanno.

- **Modularità:** presenza di moduli distinti per la specializzazione e composizione del lavoro.
- **Regolarità:** si realizzano strutture simili e ripetibili invece di fare sempre le cose in modo diverso.

Si usano moduli più piccoli per svolgere azioni più grandi quando combinati.

4 Circuiti Digitali

4.1 Sistema binario

Noi utilizziamo il sistema decimale, ovvero un sistema **posizionale**, dove ogni cifra possiede un valore diverso a seconda della sua posizione.

Anche il **sistema binario** è un sistema posizionale, ma le cifre sono solo 2 (0,1).

- **Esempio decimale:**

$$13_{10} = (1 \cdot 10^1) + (3 \cdot 10^0)$$

- **Esempio binario:**

$$10_2 = (1 \cdot 2^1) + (0 \cdot 2^0) = 2_{10}$$

Nei computer si utilizza il sistema binario per via della sua semplicità e della sua compatibilità con i moduli logici, per esempio una moltiplicazione è più facile da svolgere in binario tramite un modulo **AND**, invece che in decimale.

4.2 Numeri negativi

Per rappresentare i numeri negativi in binario ci sono due modi:

- **Modulo e segno:** si riserva il **MSB** (Most Significant Bit, più a sinistra) per il segno (1 se negativo, 0 se positivo).
- **Complemento a 2:** nato perché complicate le operazioni con modulo e segno (if else); si ha un algoritmo per ottenere il binario di un numero negativo:
 1. Si trasforma il numero in binario.
 2. Si fa il NOT di ogni bit
 3. si aggiunge "+1"

Su un byte di solito si possono realizzare **256** possibili combinazioni positive, ma col complemento a 2 l'intervallo si spezza:

- 128 valori positivi (0 - 127)
- 128 valori negativi (-128 - -1)

Nel modulo e segno addirittura ci sono 2 rappresentazioni diverse per lo 0, altro motivo per cui è stato introdotto il complemento a 2.

Ecco come funziona veramente il complemento a 2:

- Il MSB vale un valore negativo (-2^7)
- Tutti gli altri bit valgono valori positivi
- Se si ricalcola la somma di un complemento a 2 per trovare il suo valore decimale, si ottiene il numero negativo.

Per esempio, calcoliamo -10:

$$10_{10} = 00001010_2$$

$$\neg(00001010) = 11110101$$

$$11110101 + 1 = 11110110$$

$$11110110_2 = (-2^7 + 2^6 + 2^5 + 2^4 + 2^2 + 2)_{10} = (-128 + 64 + 32 + 16 + 4 + 2) = -10_{10}$$

ATTENZIONE: se si effettua una somma tra numeri in complemento a 2 che hanno segni concordi e si ottiene un risultato con segno diverso allora significa che c'è stato un **overflow**.

4.3 Numeri con la virgola (Floating Point)

Il **floating point (FP)** permette di rappresentare i numeri con cifre decimali. Ecco la sua struttura:

$$[\text{segno}][\text{esponente}][\text{mantissa}]$$

Questa struttura può essere lunga 32 o 64 bit.

- **Segno:** 1 bit, segno (+/-) del numero.
- **Esponente:** 8 o 11 bit, esponente per cui "10" è moltiplicato il numero decimale, per esempio:

$$10,5 = 105 \cdot 10^{(-1)}$$

- **Mantissa:** 23 o 52 bit, contiene il numero intero da moltiplicare al "10" ().

4.4 Caratteri (ASCII)

Per rappresentare i caratteri in binario si utilizza la **codifica ASCII**, che utilizza **ottetti** (7/8 bit) per rappresentare ogni carattere.

Ogni carattere è identificato da un **ID**. Offre:

- 26 lettere maiuscole
- 26 lettere minuscole
- 10 cifre
- Punteggiatura
- Caratteri speciali (invio, tab, DEL)

Vediamo i range di ID che ci interessano:

- **Non stampabili:** 0 – 31, 127
- **Stampabili:** 32 – 126
- **Cifre:** 48 – 57
- **Lettere maiuscole:** 65 – 90

- **Lettere minuscole:** 97 – 122
- **Spazio:** 32

Come si può notare, ogni lettera maiuscola è sempre alla stessa distanza dalla sua corrispondente minuscola: per questo basta aggiungere "32" ad una lettera maiuscola (o sottrarlo alla minuscola) per switchare tra lowercase e uppercase.

4.5 Notazione per grandezza input/output

Per indicare quanti bit è grande un input o output, si usa questa notazione:

$$n$$

$$\rightarrow$$

4.6 Tempi per calcolare risultati

Quando realizziamo circuiti digitali bisogna tenere di conto quanto ci mettono le porte a inicializzarsi e quanto ci mette il risultato a essere emesso.

Indichiamo il ritardo di una porta da massimo **8 ingressi** con:

$$\Delta t$$

Si considera il ritardo per le porte di AND e OR, il tempo della porta NOT è talmente insignificante che non viene considerato ($= 0$).

Nei nostri circuiti si usano sempre porte da 8.

Per realizzare porte da 9 o più è necessario collegare più porte insieme e farle congiungere ad una porta unica.

Quando bisogna realizzare circuiti con tanti ingressi si tende a realizzare una struttura ad albero.

Ecco come si trova il tempo di impiego di un modulo:

1. Si crea la tabella di verità con a sinistra gli input e a destra gli output
2. Si scrivono tutte le righe possibili degli input (combinazioni, 2^n , n = numero di input diversi)
3. Si calcola il risultato a seconda del modulo e si scrive per ogni combinazione
4. Ripetere l'istruzione precedente per ogni output
5. Una volta trovata la tabella completa, si trova l'espressione per calcolare ogni output: si eguaglia un output alla somma dei prodotti dei vari input delle combinazioni che hanno risultato un output = "1" (espressione logica, i bit = 0 sono "negati", ovvero con la overline (NOT))
6. Si contano termini (OR) e variabili (AND), spesso ogni modulo li usa entrambi, arrivando al minimo a $2\Delta t$, però se uno tra i due supera il numero 8 allora si conta un tempo in più.

Spesso le formule si possono semplificare, ma ciò non influisce il tempo impiegato dai moduli, bensì ci può permettere di risparmiare materiale e spazio.

5 Esadecimale

Oltre al sistema binario, spesso viene utilizzato anche il sistema **esadecimale**, ovvero in base 16.

Questo sistema è comodo, basta pensare al fatto che 16 in binario necessita di esattamente 4 bit (un **nibble**), questo vuol dire che ogni 4 bit si ha una cifra in esadecimale e viceversa. Ecco le cifre:

1, 2, 3, 4, 5, 6, 7, 8, 9, *A*, *B*, *C*, *D*, *E*, *F*

Con:

$$A = 10, F = 15$$

La base esadecimale viene introdotta da un "**0x**", così:

0x5AF

5.1 Processo di progettazione dei circuiti digitali

Ecco il procedimento:

1. Descrizione a parole della richiesta
2. Realizzazione pseudo codice
3. Approccio strutturale, composizione componenti
4. Tabella di verità
5. Somma di prodotti (espressione)
6. Rete di porte
7. Tempo di stabilizzazione

Esempio : dato x da n bit:

```
if ( pari(x) ) {  
    x++;  
} else {  
    x--;  
}
```

Per trovare i pari basta notare che tutti i numeri pari in binario hanno il LSB (Least Significant Bit) segnato a "0".

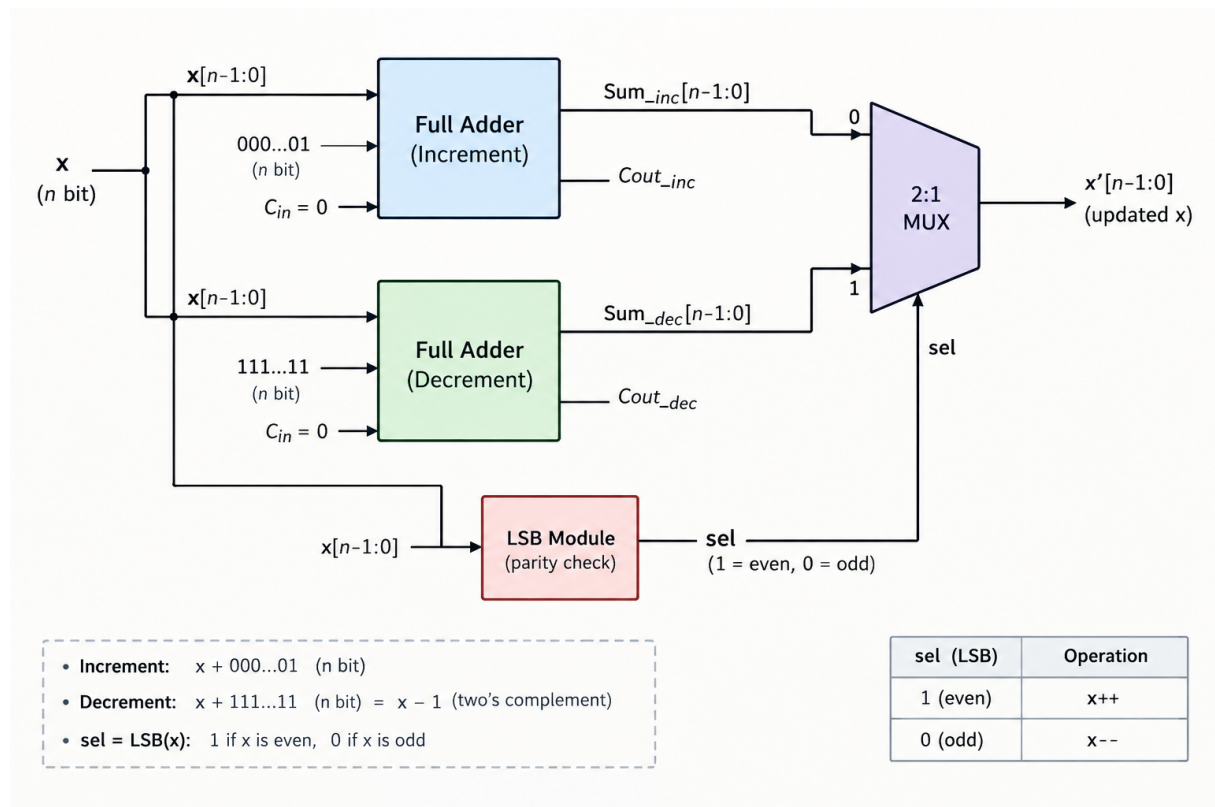


Figure 1: Circuito Pari-Dispari

5.2 Mappe di Karnaugh

Modo alternativo alle tabelle di verità:

	$x = 0$	$x = 1$
$y = 0$	0	0
$y = 1$	0	0

	$x_1x_0 = 00$	$x_1x_0 = 01$	$x_1x_0 = 11$	$x_1x_0 = 10$
$y_1y_0 = 00$	0	0	0	0
$y_1y_0 = 01$	0	0	0	0
$y_1y_0 = 11$	0	0	0	0
$y_1y_0 = 10$	0	0	0	0

Come si può vedere viene prima "11" che di "10" perché si usa il **codice Gray**, il quale indica che ad ogni passaggio (celle adiacenti) può cambiare un solo bit.

6 Moduli

Questa sezione elenca vari esempi di moduli per le operazioni del calcolatore.

6.1 Sommatore

Semplice modulo che somma due numeri:

- Riporto iniziale di 0
- Un output unico

6.2 Sottrattore (sommatore in complemento a 2)

Il **sottrattore** è un sommatore che:

- Ha un riporto iniziale di 1
- Effettua il not sui bit

6.3 Comparatore

Ha due numeri in input e restituisce un valore booleano a seconda dell'uguaglianza dei due numeri immessi. Denotato da una scatola con "=" scritto dentro.

6.4 Codificatore

Ha un input e un output, restituisce la posizione dell'1 nell'input. Denotato da una scatola con "COD" scritto dentro.

6.5 Moduli per algebra booleana

Per svolgere i controlli booleani si utilizza una combinazione (circuito) dei seguenti moduli:

- Modulo AND
- Modulo OR
- Modulo NOT
- Modulo XOR
- Modulo NAND

Tutti prendono due o più input in entrata e restituiscono un output ad eccezione del NOT, che prende un solo input.

Per rappresentare i circuiti si usano delle **espressioni**, vediamo delle notazioni di algebra booleana in tale progettazione:

- Quando un parametro ha un "overline" come $\bar{x}$ allora significa che passa da un NOT.

- La moltiplicazione tra variabili corrisponde all'AND, dato che se facciamo la moltiplicazione tra più valori booleani il risultato sarà diverso da 0 solo se tutti i membri sono settati a "1".
- La somma tra variabili corrisponde all'OR, dato che il risultato è non nullo solo se almeno uno dei membri è settato a "1".

Nota bene: un qualsiasi valore positivo, anche se diverso da "1", viene contato come **true**. Vediamo un esempio:

$$\bar{x}yz + x\bar{y}z + xy\bar{z} + xyz$$

Questo è un circuito con 4 moduli AND, 3 moduli OR e 3 moduli NOT realizzato in questo modo:

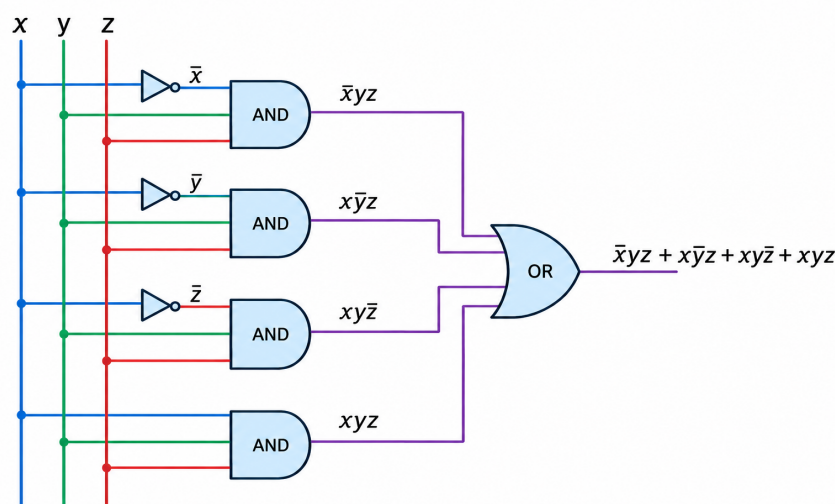


Figure 2: Circuito logico

6.6 Modulo di scelta

Alcune volte è necessario effettuare delle scelte a seconda di un parametro, esiste un modulo fatto su misura che:

- Prende in input 2 scelte
- Analizza un terzo input come "condizione"
- Sceglie in base all'analisi effettuata

Questo modulo è strettamente collegato ai moduli dell'algebra booleana.

6.7 Full Adder

Sommatore speciale che:

- Ha 2 input

- Ha 1 output risultato
- Ha 1 output di riporto (che va al prossimo FA)
- Ha 1 riporto iniziale (anche nullo)

X	Y	C_{in}	Ris	Rip
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 1: Tabella di verità del Full Adder

$$Ris = X \cdot \overline{Y} \cdot \overline{C_{in}} + \overline{X} \cdot Y \cdot \overline{C_{in}} + \overline{X} \cdot \overline{Y} \cdot C_{in} + X \cdot Y \cdot C_{in}$$

$$Rip = X \cdot Y + X \cdot C_{in} + Y \cdot C_{in}$$

Il Full Adder può sommare anche numeri in base 2 se combinato con un multiplexer. Il suo tempo di impiego corrisponde a $4\Delta t$.

6.8 Multiplexer

Il **multiplexer** racchiude un circuito, ha solo input e output visibili dall'esterno.

I_0	I_1	S	Y
0	0	0	0
0	1	0	0
1	0	0	1
1	1	0	1
0	0	1	0
0	1	1	1
1	0	1	0
1	1	1	1

Table 2: Tabella di verità del Multiplexer 2:1

$$Y = \overline{S} \cdot I_0 + S \cdot I_1$$

6.9 Demultiplexer

Contrario del multiplexer, ha 1 input e 2 porte in uscita.

Trasmette l'input in una porta in uscita a seconda del **bit di controllo** in input e azzerla la porta in uscita non utilizzata.

C	X	z_1	z_0
0	0	0	0
0	1	1	0
1	0	0	0
1	1	0	1

Table 3: Tabella di verità del Demultiplexer 1:2

$$z_1 = C \cdot \overline{X}$$

$$z_0 = C \cdot X$$

Impiega un tempo di Δt (fa solo AND).